

UNIVERSIDAD EAFIT
ESCUELA DE CIENCIAS APLICADAS E INGENIERÍA
ARQUITECTURA DE SOFTWARE 2026

Tutorial 06: El Primer Estrangulamiento

Migrando el Servicio de Compras de Django a Flask (v2)

Nicolás Rodríguez

Profesor: Nicolás Ramírez Vélez

30 de septiembre de 2026

1. Entregables en AWS

1.1. Prueba de Coexistencia (v1)

Postman: GET `http://IP-AWS/api/v1/productos/`

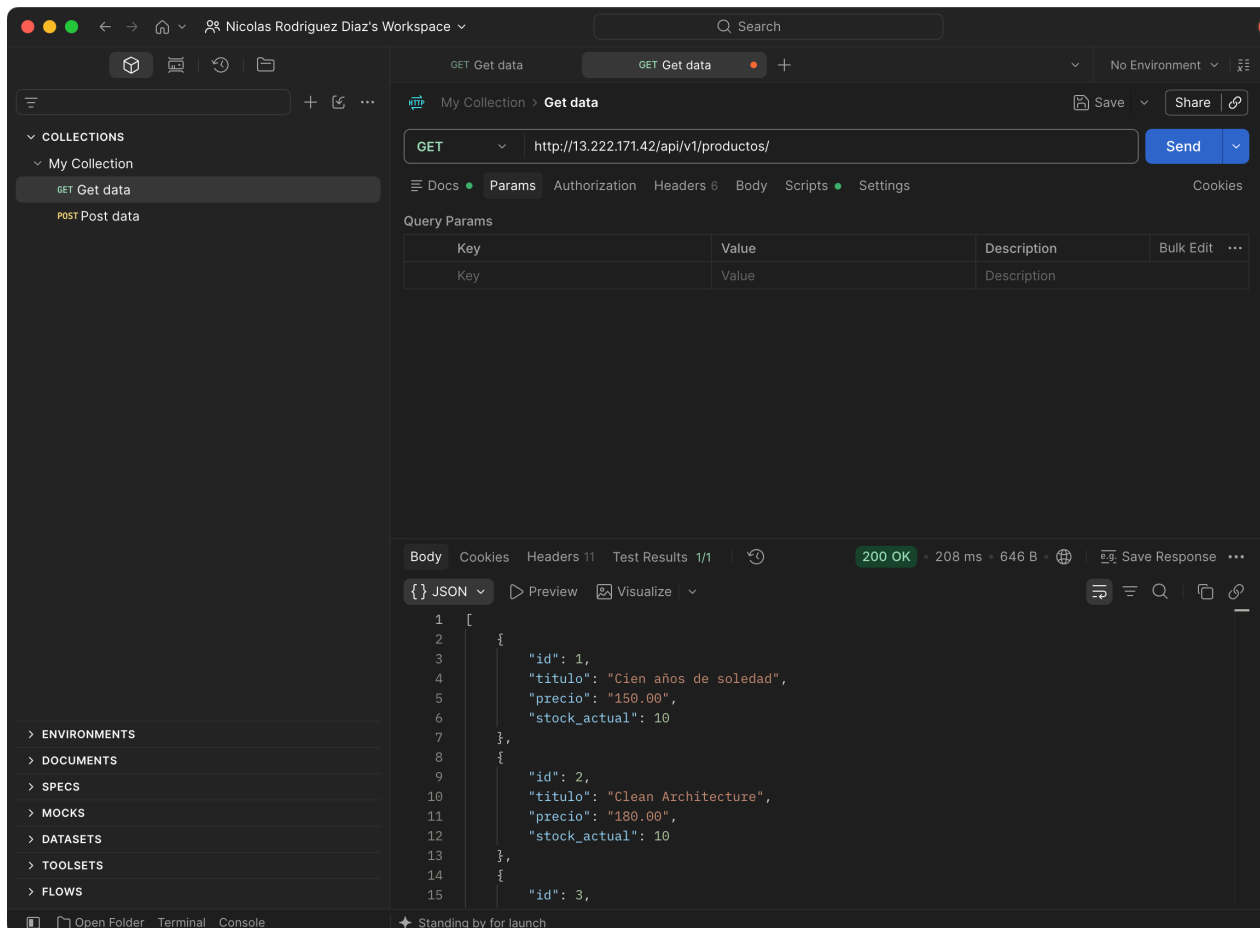


Figura 1: Postman: GET `http://13.222.171.42/api/v1/productos/` → 200 OK. Responde Django (el monolito) con el catálogo.

1.2. Prueba de Estrangulamiento (v2)

Postman: POST `http://IP-AWS/api/v2/comprar`

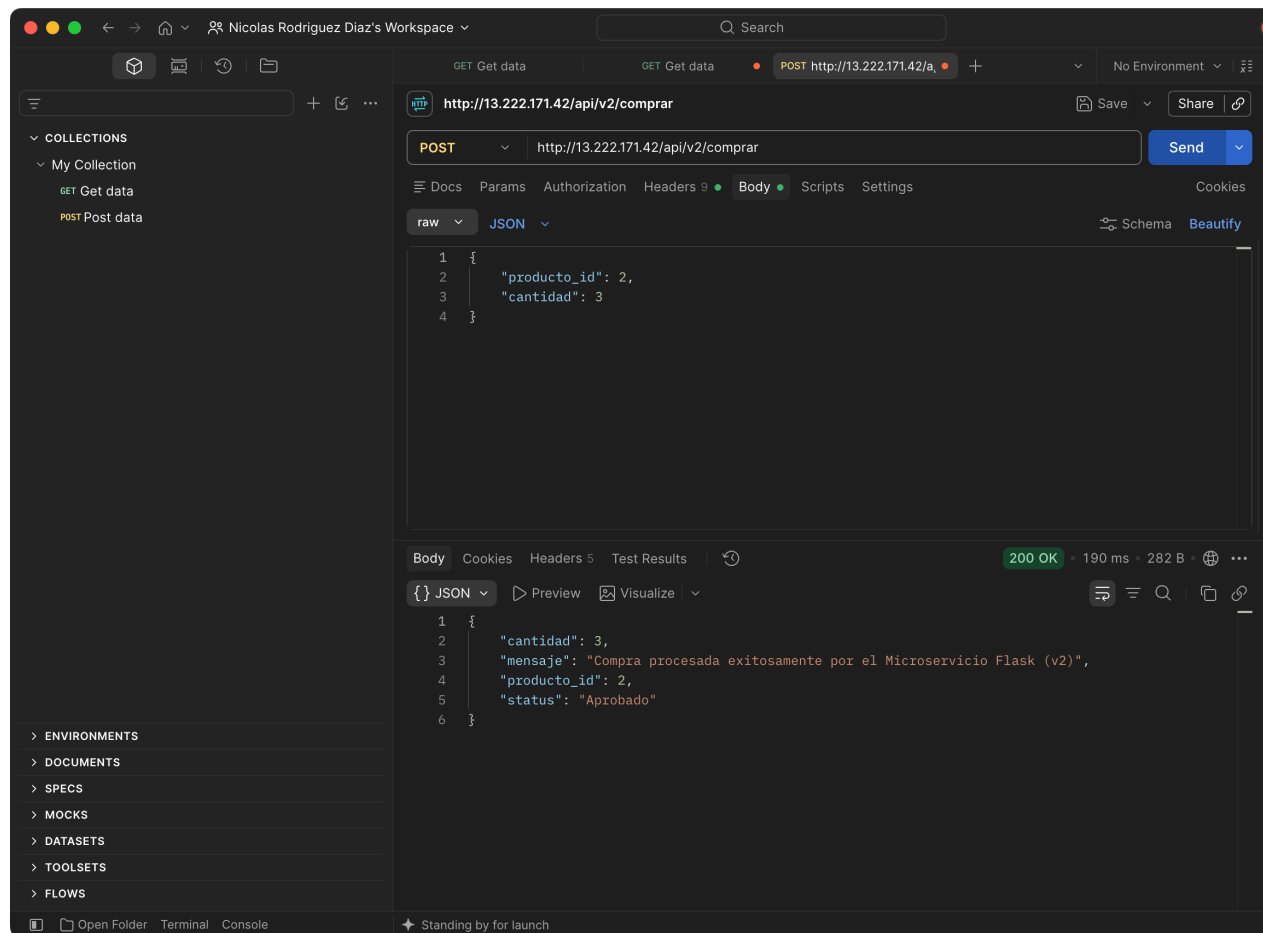


Figura 2: Postman: POST `http://13.222.171.42/api/v2/comprar` con `{ "producto_id": 2, "cantidad": 3 }` → 200 OK "Compra procesada exitosamente por el Microservicio Flask (v2)".

1.3. Logs de Orquestación

`docker compose logs nginx`

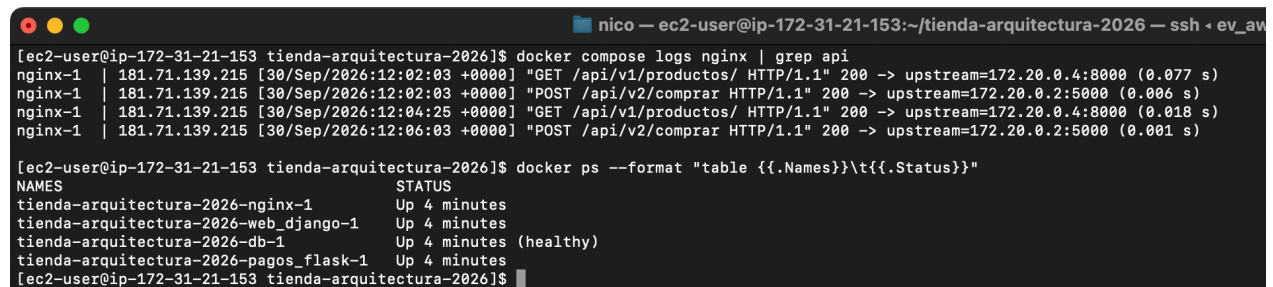


Figura 3: docker compose logs nginx en EC2: `/api/v1/` → 172.20.0.4:8000 (Django) y `/api/v2/comprar` → 172.20.0.2:5000 (Flask).

```
docker inspect en EC2 (IP interna de cada contenedor)
```

```
/tienda-arquitectura-2026-nginx-1 -> 172.20.0.5  
/tienda-arquitectura-2026-web_django-1 -> 172.20.0.4  
/tienda-arquitectura-2026-db-1 -> 172.20.0.3  
/tienda-arquitectura-2026-pagos_flask-1 -> 172.20.0.2
```

Cruce: 172.20.0.4 es `web_django` y 172.20.0.2 es `pagos_flask`. Mismo Nginx, dos backends distintos: el estrangulamiento funcionando en la nube.

2. Implementación

2.1. Paso 1: `microservicio_pagos/app.py`

```
microservicio_pagos/app.py
```

```
from flask import Flask, request, jsonify  
import os  
  
app = Flask(__name__)  
  
@app.route('/api/v2/comprar', methods=['POST'])  
def realizar_compra():  
    data = request.get_json(silent=True) or {}  
  
    # Simulacion de logica de negocio extraida  
    producto_id = data.get('producto_id')  
    cantidad = data.get('cantidad', 1)  
  
    if not producto_id:  
        return jsonify({"error": "Falta el ID del producto"}), 400  
  
    return jsonify({  
        "mensaje": "Compra procesada exitosamente por el Microservicio Flask (v2)",  
        "producto_id": producto_id,  
        "cantidad": cantidad,  
        "status": "Aprobado"  
    }), 200  
  
if __name__ == '__main__':  
    app.run(host='0.0.0.0', port=5000)
```

Único cambio: `request.get_json(silent=True) or {}`. Con el `get_json()` original, un POST sin cuerpo JSON revienta con un error 415 antes de llegar a la validación; así responde el 400 "Falta el ID del producto" que el propio código pretende.

2.2. Paso 2: `microservicio_pagos/Dockerfile`

```
microservicio_pagos/Dockerfile
```

```
FROM python:3.11-alpine  
WORKDIR /app  
RUN pip install flask gunicorn  
COPY . .  
EXPOSE 5000  
CMD ["gunicorn", "app:app", "--bind", "0.0.0.0:5000"]
```

2.3. Paso 3: docker-compose.yml

docker-compose.yml

```
services:
  db:
    image: postgres:15
    volumes:
      - postgres_data:/var/lib/postgresql/data
    environment:
      - POSTGRES_DB=tienda_db
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=secret_password
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d tienda_db"]
      interval: 3s
      timeout: 3s
      retries: 20

  web_django: # El Monolito (v1)
    build: .
    command: sh -c "python manage.py migrate && python manage.py seed_tienda && python manage.py collectstatic --noinput && gunicorn Tienda.wsgi:application --bind 0.0.0.0:8000"
    volumes:
      - ./app
    depends_on:
      db:
        condition: service_healthy
    environment:
      - DB_NAME=tienda_db
      - DB_USER=postgres
      - DB_PASSWORD=secret_password
      - DB_HOST=db
      - DB_PORT=5432

  pagos_flask: # El Microservicio (v2) - EL ESTRANGULADOR
    build: ./microservicio_pagos
    ports:
      - "5000:5000" # Solo para pruebas iniciales, luego lo cerraremos

  nginx:
    image: nginx:1.25
    ports:
      - "80:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf
    depends_on:
      - web_django
      - pagos_flask

volumes:
  postgres_data:
```

2.4. Paso 4: nginx/nginx.conf

```

                                nginx/nginx.conf
# Registro que muestra a QUE backend se envió cada petición (evidencia del Strangler)
log_format strangler '$remote_addr [$time_local] "$request" $status '
                        '-> upstream=$upstream_addr ($upstream_response_time s)';

upstream django_v1 {
    server web_django:8000;
}

upstream flask_v2 {
    server pagos_flask:5000;
}

server {
    listen 80;
    access_log /var/log/nginx/access.log strangler;

    location = /favicon.ico { access_log off; log_not_found off; }

    # RUTA VIEJA: Sigue yendo a Django
    location /api/v1/ {
        proxy_pass http://django_v1;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    # RUTA NUEVA: Estrangulamos la función de compra y la mandamos a Flask
    location /api/v2/comprar {
        proxy_pass http://flask_v2;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    # Todo lo que aun no se ha migrado (HTML, admin, estaticos) sigue en el monolito
    location / {
        proxy_pass http://django_v1;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}

```

Además de las dos `location` del enunciado, dejé `location /` apuntando a Django: el HTML, el admin y los estáticos aún no se han migrado, y el Strangler Pattern dice que lo no estrangulado sigue en el legado. Sin esa regla, Nginx devolvería 404 a todo lo demás.

2.5. Endpoint v1 de productos en Django

La prueba de coexistencia pide `GET /api/v1/productos/`, que el monolito no tenía. Lo agregué como `ListAPIView` reutilizando el `LibroSerializer` del Tutorial 03 (lectura pura: aquí sí basta una vista genérica, sin Service Layer), con su prueba unitaria.

```

                                tienda_app/api/views.py

class LibroListAPIView(ListAPIView):
    queryset = Libro.objects.select_related('inventario').order_by('id')

```

```
serializer_class = LibroSerializer

# tienda_app/urls.py
path('api/v1/productos/', LibroListAPIView.as_view(), name='api_productos'),
```

3. Paso 5: Despliegue en AWS

Terminal EC2

```
cd tienda-arquitectura-2026
# (codigo actualizado subido por scp; con GitHub seria: git pull origin main)
docker compose up -d --build
docker ps                # db, web_django, pagos_flask, nginx en Up
docker compose logs nginx # evidencia 1.3
```

Si aparece 502 Bad Gateway o un 404 en v2, el anexo *Guía de Supervivencia* aplica tal cual: `docker compose logs pagos_flask`, revisar que la `location` coincida con la ruta de Flask (sin slash final en ambos) y `docker compose restart nginx` tras cambiar el `.conf`.

Anexo: validación local de la arquitectura (mismo docker-compose)

Probé el despliegue completo en una VM Linux local (Colima, IP 192.168.64.2) con los cuatro contenedores: `db`, `web_django`, `pagos_flask` y `nginx`.

Coexistencia: v1 sigue en Django

```
$ curl -i http://192.168.64.2/api/v1/productos/
HTTP/1.1 200 OK
Server: nginx/1.25.5
Content-Type: application/json

[{"id":1,"titulo":"Cien años de soledad","precio":"150.00","stock_actual":10}, {"id":2,"titulo":"Clean Architecture","precio":"180.00","stock_actual":9}, {"id":3,"titulo":"Patrones de Diseño (GoF)","precio":"165.00","stock_actual":9}, {"id":4,"titulo":"Domain-Driven Design","precio":"210.00","stock_actual":10}]
```

Figura 4: GET `/api/v1/productos/` → 200 OK con el catálogo (id, título, precio, `stock_actual`) servido por Django a través de Nginx.

Estrangulamiento: v2 ya es Flask

```
$ curl -i -X POST http://192.168.64.2/api/v2/comprar -H "Content-Type: application/json" -d '{"producto_id": 2, "cantidad": 3}'
HTTP/1.1 200 OK
Server: nginx/1.25.5
Content-Type: application/json

{"cantidad":3,"mensaje":"Compra procesada exitosamente por el Microservicio Flask (v2)","producto_id":2,"status":"Aprobado"}
```

Figura 5: POST `/api/v2/comprar` → 200 OK: "Compra procesada exitosamente por el Microservicio Flask (v2)".

Logs de Nginx: dos backends distintos

Configuré un `log_format` propio que agrega `$upstream_addr`, así el log dice explícitamente a qué contenedor fue cada petición:

```
$ docker compose logs nginx | grep api
192.168.64.1 [30/Sep/2026:10:59:43 +0000] "GET /api/v1/productos/ HTTP/1.1" 200 -> upstream=172.18.0.4:8000 (0.158 s)
192.168.64.1 [30/Sep/2026:10:59:49 +0000] "POST /api/v2/comprar HTTP/1.1" 200 -> upstream=172.18.0.3:5000 (0.017 s)
```

Figura 6: docker compose logs nginx: v1 → 172.18.0.4:8000, v2 → 172.18.0.3:5000.

`docker inspect` (IP de cada contenedor en la red interna)

```
/proyectotienda-tutoriales-nginx-1 -> 172.18.0.5
/proyectotienda-tutoriales-web_django-1 -> 172.18.0.4
/proyectotienda-tutoriales-db-1 -> 172.18.0.2
/proyectotienda-tutoriales-pagos_flask-1 -> 172.18.0.3
```

Cruce: 172.18.0.4 es `web_django` (Gunicorn :8000) y 172.18.0.3 es `pagos_flask` (Gunicorn :5000). La misma fachada, dos cerebros distintos, y el cliente no se entera.